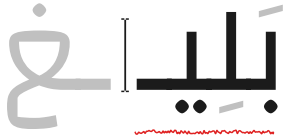
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Ahmed Hamed Gaber Hamed **Akram Hany Karam Salam**
Amir Anwar Bekhit Awd Kedis **Somia Saad Ismail El-Shemy**

Supervised by

Dr. Ayman AboElhassan

July, 2025

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, and text-editing models. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

الملخص

لا تزال الكتابة الرقمية باللغة العربية تعاني من نقص الأدوات المتكاملة التي تستطيع تصحيح نصوص العربية الفصحى بدقة، والعمل بزمن استجابة مناسب، وتقديم تفسير واضح يساعد المستخدم على التعلّم بدل الاكتفاء بإظهار التصحيح فقط. فكثير من الأدوات المتاحة تركز على الأخطاء الإملائية السطحية أو تقدم اقتراحات غير مفسرة. يعالج مشروع بليغ هذه المشكلة من خلال بناء مساعد كتابة عربي يجمع بين اكتشاف الأخطاء النحوية، وتصحيحها، ومعالجة الأخطاء الإملائية، واقتراح الكلمة التالية، وتقديم تفسير مبسط للقاعدة أو سبب الخطأ داخل نظام واحد موجه للمستخدم. ويهدف المشروع إلى خدمة الطلاب والمعلمين وصناع المحتوى العربي عبر تحسين جودة الكتابة وجعل عملية التصحيح أكثر فائدة من الناحية التعليمية.

يعتمد النظام المنفذ على بنية معيارية تبدأ بطبقة للمعالجة المسبقة تشمل التطبيع، وتحديد حدود الكلمات، والتجزئة، ثم التحليل الصرفي وفك الالتباس. بعد ذلك يدمج بليغ محركاً لاكتشاف الأخطاء النحوية يعتمد على المزج بين قواعد لغوية، وكشافات معجمية، ونماذج تعلم آلي، ثم يمرر النتائج إلى وحدات التصحيح المبنية على الاستدلال المعتمد على الأنطولوجيا، والبحث المعجمي، ونماذج التحرير النصي. وبالتوازي مع ذلك، يوفر النظام وحدةً لاقتراح الكلمة التالية تشمل التنبؤ بالكلمة التالية والإكمال التلقائي للكلمة الجارية كتابتها. وتعرض المنصة النهائية الأخطاء والتصحيحات والاقتراحات والتفسيرات اللغوية بصورة تفاعلية.

تشمل مخرجات المشروع خط معالجة مسبقة متكاملاً، وخدمات لاكتشاف الأخطاء وتصحيحها، ووحدة اقتراح الكلمات، وتطبيقاً عملياً للمستخدم. واعتمد التحقق من النظام على اختبارات آلية للوحدات والتكامل، إضافة إلى تقييم مبني على موارد عربية معروفة مثل QALB14 وQALB15 وZAEBUC. وتُظهر المحصلة أن بليغ يقدم نموذجاً عملياً لمساعد كتابة عربي قابل للتفسير يجمع بين المعالجة اللغوية والتعلم الآلي لخدمة الكتابة العربية بصورة أكثر جودة وفائدة.

Acknowledgment

Baligh was not built through technical effort alone, but through the support, patience, and trust of many people around us. We would first like to thank our supervisor, Dr. Ayman AboElhassan, for his steady guidance throughout this project. His questions, feedback, and encouragement pushed us to think more carefully, work more seriously, and keep improving the project whenever we felt uncertain or stuck.

We are also deeply grateful to the Faculty of Engineering at Cairo University and the Department of Computer Engineering. The years we spent there shaped the way we think, learn, and solve problems, and this project reflects much of what we gained from that journey. Baligh became possible because of the academic foundation and sense of responsibility that this environment gave us.

We would also like to thank one another as a team. This project demanded time, patience, repeated trial and error, and many long discussions before ideas became clear and features became real. What made that process meaningful was not only the final result, but the way we supported each other through the difficult parts, shared the pressure, and kept moving forward together.

Finally, we thank our families and loved ones with all sincerity. Their patience during busy and stressful periods, their constant encouragement, and their belief in us gave us the strength to continue. Any achievement in this work carries a part of their support within it, and for that we are truly grateful.

Contents

List of Figures

List of Tables

List of Abbreviations

Put abbreviations in alphabetical order.

Abbreviation	Meaning
AI	Artificial Intelligence
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction

List of Symbols

Put symbols in alphabetical order. Greek symbols should come first, followed by English symbols.

Symbol	Meaning
α	Learning rate or weighting coefficient
P	Probability
T	Time or sequence length

Contacts

Team Members

Name	Email	Phone Number
Ahmed Hamed Gaber Hamed	ahmed.hamed03@eng-st.cu.edu.eg	01226968657
Akram Hany Karam Salam	akram.sallam03@eng-st.cu.edu.eg	01148746563
Amir Anwar Bekhit Awd Kedis	amir.awd03@eng-st.cu.edu.eg	01221461992
Somia Saad Ismail El-Shemy	somia.elshemy02@eng-st.cu.edu.eg	01080732462

Supervisor

Name	Email	Phone Number
Dr. Ayman AboElhassan	ayman.abo.elmaaty@eng.cu.edu.eg	01060836189

Chapter 1: Introduction

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. This chapter introduces the motivation behind the project, defines the problem it addresses, summarizes the main outcomes delivered by the project, and outlines the organization of the remainder of the report.

1.1. Motivation and Justification

Arabic digital writing still lacks broadly available tools that combine grammatical support, writing speed, and educational value in a single user experience. While many writing assistants for other languages provide immediate correction, prediction, and revision support, Arabic users often rely on tools that address only isolated spelling mistakes, provide limited grammatical help, or return suggestions without clarifying the linguistic reason behind them. This gap is especially important in Modern Standard Arabic, where accurate writing is influenced by rich morphology, orthographic ambiguity, and context-sensitive grammatical structure.

These characteristics make Arabic NLP more demanding to model and more difficult to serve well in real time. A useful writing assistant must do more than detect surface-level errors; it must process text carefully, understand partially typed input, and return feedback quickly enough to support natural writing. It must also present corrections in a way that is understandable to users rather than functioning as a black box.

Baligh is motivated by this need for an integrated Arabic writing assistant that combines grammatical error detection, grammatical error correction, next-word assistance, and explanatory feedback in one workflow. The project is intended to benefit students, educators, and Arabic content writers who need writing support that improves both text quality and user understanding at the same time.

1.2. Project Objectives and Problem Definition

The primary objective of Baligh is to build an explainable writing assistant for Modern Standard Arabic that can help users write more accurately and efficiently. To achieve this objective, the project integrates a preprocessing pipeline for normalization,

segmentation, and morphological analysis with a grammatical error detection subsystem, grammatical error correction components, a next-word suggestion module, and a user-facing application interface. The system is designed to highlight issues, generate ranked suggestions, provide next-word and auto-completion support, and present explanations that make the feedback more useful for learning.

In addition to functional assistance, the project aims to demonstrate that Arabic writing support can be delivered through a modular architecture that balances linguistic processing, machine learning, usability, and responsiveness. This makes Baligh not only a correction tool, but also a platform for combining multiple Arabic NLP capabilities into a practical workflow.

Formally, the problem addressed by this project is the lack of an integrated and explainable system for Modern Standard Arabic writing that can detect and correct grammatical issues, assist users during text entry, and deliver reliable feedback in a form suitable for real-time interactive use.

1.3. Project Outcomes

The project produced a working set of interconnected software components that together form the Baligh writing assistant. The implemented outcomes include a preprocessing pipeline responsible for text normalization, word-boundary handling, segmentation, and morphological analysis, as well as a grammatical error detection subsystem that combines rule-based, lexicon-based, and machine-learning detectors through a fusion layer.

The project also delivered grammatical error correction components that use ontology-based reasoning, dictionary-based lookup, and text-editing approaches to generate and rank correction candidates. In parallel, Baligh includes a next-word suggestion subsystem that supports both next-word prediction and word auto-completion for interactive writing scenarios.

At the application level, the project provides a backend API and a web-based client that expose these capabilities to the user through an integrated writing workflow. Testing and evaluation were supported through automated checks and experiments on established Arabic resources, including QALB14, QALB15, and ZAEBUC, in addition to the accompanying project documents and technical materials.

1.4. Document Organization

The remainder of this report is organized as follows. Chapter 2 presents the Market Visibility Study, which examines the intended users of the system, reviews related products, and discusses the practical need for Baligh.

Chapter 3 provides the Literature Survey, covering the Arabic NLP background and the research context most relevant to the project. Chapter 4 describes the System Design and Architecture, including the main modules, their responsibilities, and the overall data flow.

Chapter 5 discusses System Testing and Verification, explaining how the implemented components were checked and evaluated. Finally, Chapter 6 presents the Conclusions and Future Work, summarizing the project contributions and outlining possible directions for extending Baligh in later stages.

Chapter 2: Market Visibility Study

This chapter surveys the market related to the project and discusses similar tools or platforms. It should explain their limitations and justify the need for Baligh.

2.1. Targeted Customers

Mention the intended customers/users and how they benefit from the project.

2.2. Market Survey

List competing or related products. Discuss each one in a subsection and explain its pros and cons.

2.2.1. Competitive Product 1

Describe the product, strengths, weaknesses, and relevance to Baligh.

2.2.2. Competitive Product 2

Describe the product, strengths, weaknesses, and relevance to Baligh.

2.3. Business Case and Financial Analysis

Describe the possible business case, expected users, pricing assumptions, CapEx, OpEx, revenue, cash flow, and break-even point.

Table 2.1: *Sample Five-Year Financial Projection*

Year	Expected Users	Revenue	Expenses	Profit Before Tax
1	–	–	–	–
2	–	–	–	–
3	–	–	–	–
4	–	–	–	–
5	–	–	–	–

Chapter 3: Literature Survey

This chapter summarizes the Arabic NLP background most relevant to Baligh and reviews the detection, correction and prediction approaches that influenced the system design.

3.1. Background on Arabic NLP

Arabic writing assistance is difficult because Arabic is morphologically rich, often undiacritized, and split across several language varieties [1, 2]. In Modern Standard Arabic, clitics attach to stems and words carry substantial grammatical information, so segmentation and morphological analysis are essential preprocessing steps. Tools such as Farasa and CAMeL Tools are therefore important foundations for downstream correction and prediction [1, 3].

Ambiguity is increased by the absence of short vowels in ordinary writing and by frequent orthographic issues such as Hamza variation, Ta-Marbuta confusion, punctuation mistakes, and merge or split errors [2, 4]. This means Arabic writing support must distinguish among orthographic, morphological, and syntactic errors rather than treat everything as spelling.

The main resources for Arabic GEC are still limited compared with English. QALB-2014 and QALB-2015 remain the primary benchmarks, while ZAEBUC and Tibyan extend coverage with additional corrected data and richer annotation [5–7]. Treebanks such as PADT, CAMeL Treebank, and I3rab are also useful because they expose grammatical structure that can support detection and explanation [8–10]. For next-word assistance, efficient language modeling remains important, with KenLM offering a practical low-latency option and neural models offering stronger context modeling at higher cost [11–13].

3.2. Background on Writing Assistance Systems

Writing assistants help users during text entry, not only after writing is complete. In Arabic, this usually requires a pipeline for normalization, segmentation, error detection, correction, ranking, and feedback presentation, with next-word assistance added when interactive typing support is needed.

Arabic GEC literature mainly falls into three families. Rule-based systems use hand-crafted rules, lexicons, and regular expressions, as in SAHSOH@QALB-2015, and are attractive because they are interpretable [5]. Ontology-based systems, such as the approach of Moukrim et al., encode grammar more explicitly and support explanation-oriented correction [14]. Neural systems improve coverage and benchmark performance, especially with Transformer-based or pretrained seq2seq models [15,16]. Their main trade-off is lower interpretability and higher deployment cost.

Text-editing GEC is particularly important for Baligh. Instead of freely generating a corrected sentence, it predicts token-level edits, which makes the output faster and easier to align with user-visible changes. Alhafni and Habash showed that this approach is both accurate and efficient for Arabic [17].

Corrected	النفسية ٠ *Alnfsyħ*		الصحة *AlSHħ*			سيما *symA*	ولا *wlA*	بالصحة *bAlSHħ*				الاهتمام *AlAhtmAm*				يجب *yjb*			(a)
	7		6		5	4		3			2	1				0				
Erroneous	النفسيه *Alnfsyh*		الصحه *AlSHh*		في *fy*	ولاسيما *wlAsymA*		لصحه *lSHh*			ب *b*	الإهتمام *AlĂhtmAm*				يجب *yjb*			(b)
Word Edits	KKKKKKR_[ة]A_[.]		KKKKR_[ة]		DD	KKKI_[ ]KKKK		MI_[ا]KKKR_[ة]			K	KKR_[ا]KKKKK				KKK			(c)
Word Edits (Compressed)	K*R_[ة]A_[.]		K*R_[ة]		D*	KKKI_[ ]K*		MI_[ا]K*R_[ة]			K*	KKR_[ا]K*				K*			(d)
	7b	7a	6b	6a	5	4		3b	3a		2	1b	1a			0			
Tokenized Erroneous	ه## *##h*	النفسي *Ālnfsy*	ه## *##h*	الصح *AlSH*	في *fy*	ولاسيما *wlAsymA*		ه## *##h*	لصح *lSH*		ب	هتمام## *##htmAm*	الإ *AlĂ*			يجب *yjb*			(e)
Subword Edits	R_[ة]A_[.]	KKKKKK	R_[ة]	KKKK	DD	KKKI_[ ]KKKK		R_[ة]	MI_[ا]KKK		K	KKKKK	KKR_[ا]			KKK			(f)
Subword Edits (Compressed)	R_[ة]A_[.]	K*	R_[ة]	K*	D*	KKKI_[ ]K*		R_[ة]	MI_[ا]K*		K*	K*	K*R_[ا]			K*			(g)

Figure 3.1: *Example of text-editing tags for Arabic grammatical error correction.*

Writing assistants also need grammatical error detection and useful feedback. Treating GED as a sequence labeling task can improve interpretability, and resources such as ARETA help classify correction outputs into meaningful error types [4, 16]. For next-word assistance, n-gram models remain attractive for low latency, while neural and character-aware models offer stronger contextual modeling when more computation is acceptable [11, 12, 18]. ARWI is a strong example of a more complete Arabic writing assistant built around these ideas [19].

3.3. Comparative Study of Previous Work

Previous work shows a clear trade-off. Symbolic systems are more explainable, but neural systems usually provide broader coverage and better benchmark results. Text-editing models offer a useful compromise because they keep the correction output structured while remaining much faster than fully generative systems [5, 14, 15, 17].

The literature also shows that strong Arabic writing support depends on more than one model. Preprocessing tools such as Farasa and CAMeL Tools are needed before correction, benchmark datasets shape what systems can learn, and predictive text models address a separate but complementary part of the user experience [1, 3, 11]. The main gap is therefore integration: many papers solve one subproblem well, but fewer systems combine correction, prediction, and explanation in one practical workflow.

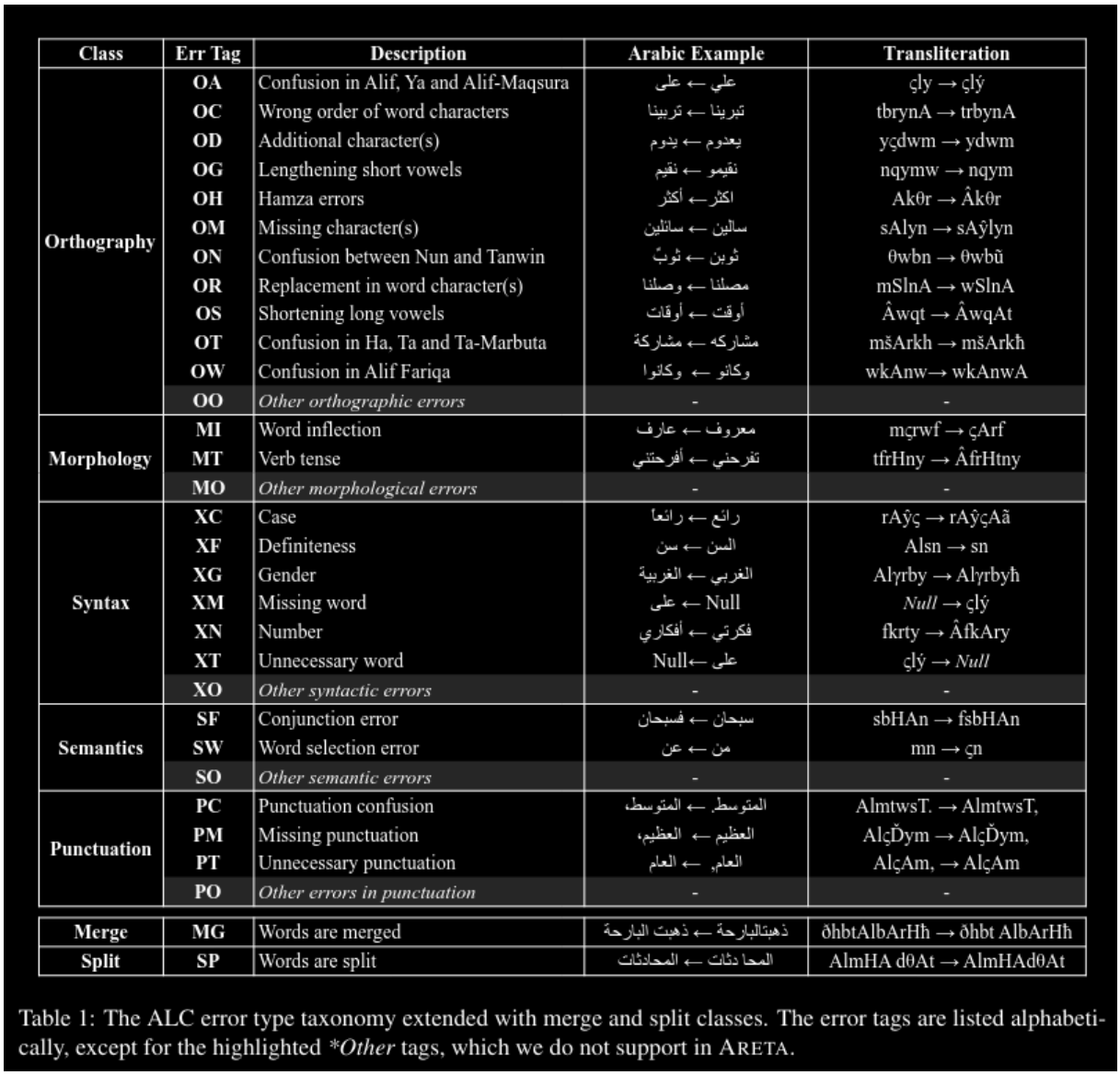
Table 3.1: *Comparison of Related Arabic Writing Assistance Approaches*

Work/System	Main Approach	Strengths	Limitations
SAHSOH@QALB-2015 [5]	Rule-based correction on QALB	Explainable and lightweight	Limited coverage
Moukrim et al. [14]	Ontology-based syntactic correction	Strong grammar grounding	Hard to scale
Solyman et al. [15]	Transformer-based Arabic GEC	Strong benchmark results	Heavier and less interpretable
Alhafni et al. [16]	Pretrained seq2seq GEC with GED input	State-of-the-art performance	Model-heavy pipeline
Alhafni and Habash [17]	Text-editing GEC	Fast and accurate	Needs integration with full product layers
ARWI [19]	Arabic writing assistant workflow	Closest system-level reference	Learner-oriented scope

3.4. Implemented Approach

The survey suggests that Baligh should not depend on a single method. A purely rule-based system is easy to explain but limited in coverage, while a purely neural system is stronger statistically but harder to interpret and deploy. Baligh therefore adopts a hybrid design that combines linguistic preprocessing, explainable correction logic, and lightweight predictive assistance.

At the front of the pipeline, Baligh uses morphology-aware preprocessing because segmentation, normalization, and disambiguation are essential for Arabic NLP [1, 3]. For correction, the literature supports a layered strategy: symbolic rules and ontology-inspired ideas remain useful for precision and explanation, while modern GEC findings, especially text-editing approaches, offer better scalability and speed [5, 14, 17].



Class	Err Tag	Description	Arabic Example	Transliteration
Orthography	OA	Confusion in Alif, Ya and Alif-Maqsura	علي ← على	ɕly → ɕlý
	OC	Wrong order of word characters	تبرينا ← تربينا	tbrynA → trbynA
	OD	Additional character(s)	يعدوم ← يدوم	yɕdwm → ydwm
	OG	Lengthening short vowels	نقيمو ← نقيم	nqymw → nqym
	OH	Hamza errors	اكثر ← أكثر	Akθr → Âkθr
	OM	Missing character(s)	سالين ← سائلين	sAlyn → sAŷlyn
	ON	Confusion between Nun and Tanwin	ثوبن ← ثوبٌ	θwbn → θwbũ
	OR	Replacement in word character(s)	مصلنا ← وصلنا	mSlnA → wSlnA
	OS	Shortening long vowels	أوقت ← أوقات	Âwqt → ÂwqAt
	OT	Confusion in Ha, Ta and Ta-Marbuta	مشاركه ← مشاركة	mšArkh → mšArkħ
	OW	Confusion in Alif Fariqa	وكانو ← وكانوا	wkAnw→ wkAnwA
	OO	*Other orthographic errors*	-	-
Morphology	MI	Word inflection	معروف ← عارف	mɕrwf → ɕArf
	MT	Verb tense	تفرحني ← أفرحتني	tfrHny → ÂfrHtny
	MO	*Other morphological errors*	-	-
Syntax	XC	Case	رائع ← رائعاً	rAŷç → rAŷçAã
	XF	Definiteness	السن ← سن	Alsn → sn
	XG	Gender	الغربي ← الغربية	Alɣrby → Alɣrbyħ
	XM	Missing word	على ← Null	*Null* → ɕlý
	XN	Number	فكرتي ← أفكاري	fkrty → ÂfkAry
	XT	Unnecessary word	Null← على	ɕlý → *Null*
	XO	*Other syntactic errors*	-	-
Semantics	SF	Conjunction error	سبحان ← فسبحان	sbHAn → fsbHAn
	SW	Word selection error	من ← عن	mn → ɕn
	SO	*Other semantic errors*	-	-
Punctuation	PC	Punctuation confusion	المتوسط، ← المتوسط	AlmtwsT. → AlmtwsT،
	PM	Missing punctuation	العظيم ← العظيم،	AlɕĎym → AlɕĎym،
	PT	Unnecessary punctuation	العام، ← العام	AlɕAm، → AlɕAm
	PO	*Other errors in punctuation*	-	-
Merge	MG	Words are merged	ذهبتالبارحة ← ذهبت البارحة	ðhbtAlbArHh → ðhbt AlbArHh
Split	SP	Words are split	المحا دثات ← المحادثات	AlmHA dθAt → AlmHAdθAt

Table 1: The ALC error type taxonomy extended with merge and split classes. The error tags are listed alphabetically, except for the highlighted **Other* tags, which we do not support in ARETA.

Figure 3.2: *Taxonomy of common Arabic grammatical and orthographic errors relevant to Baligh.*

The same reasoning applies to prediction. Since Baligh is interactive, efficient language modeling is a practical baseline for next-word suggestion, with neural models reserved for cases where extra context modeling justifies the cost [11, 12]. Separating detection, correction, and explanation is also consistent with the literature because it improves interpretability and makes user feedback more informative [4, 16].

In summary, Baligh follows the literature by combining morphology-aware preprocessing, structured error handling, explainable correction support, and efficient prediction into one real-time writing workflow.

Chapter 4: System Design and Architecture

This chapter is the main body of the report. It should answer: what has been done, and how has it been done? The discussion should flow from the overall architecture to modules and submodules.

4.1. Overview and Assumptions

Introduce the system design and list the assumptions used during design and implementation.

4.2. System Architecture

Describe the overall architecture and how modules communicate.

4.2.1. Block Diagram

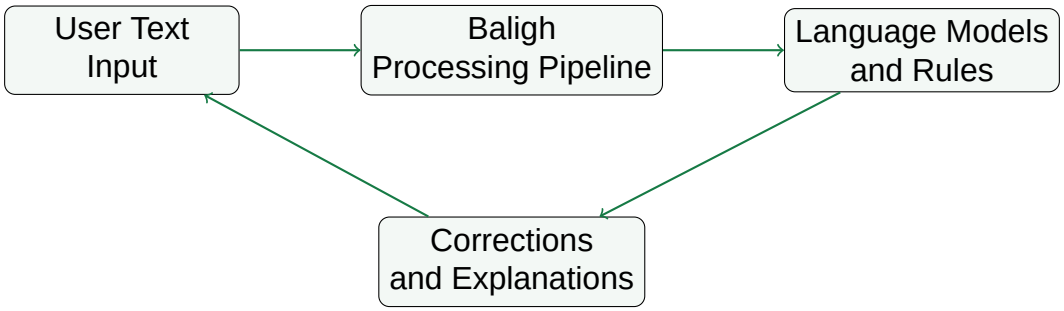


Figure 4.1: *High-level architecture of the Baligh system.*

4.3. Module 1: Grammatical Error Detection

4.3.1. Functional Description

Explain what this module does.

4.3.2. Modular Decomposition

Break the module into submodules.

4.3.3. Design Constraints

Explain constraints such as latency, Arabic ambiguity, model size, and data availability.

4.3.4. Other Description of Module 1

Add any other relevant technical details.

4.4. Module 2: Correction and Explanation Generation

4.4.1. Functional Description

Explain correction generation and explanation generation.

4.4.2. Modular Decomposition

Break the module into submodules.

4.4.3. Design Constraints

Explain constraints such as explanation quality, grammatical accuracy, and user readability.

4.4.4. Other Description of Module 2

Add any other relevant technical details.

Chapter 5: System Testing and Verification

This chapter describes how the system was tested and verified.

5.1. Testing Setup

Describe hardware, software, datasets, versions, and test environment.

5.2. Testing Plan and Strategy

Describe unit tests, integration tests, user testing, model evaluation, and manual review.

5.2.1. Module Testing

Explain how each module was tested independently.

5.2.2. Integration Testing

Explain how the full pipeline was tested after integration.

5.3. Test Schedule

Add the timeline/schedule of testing activities.

5.4. Comparative Results to Previous Work

Compare your results with baselines or previous work where applicable.

Chapter 6: Conclusions and Future Work

6.1. Faced Challenges

Discuss technical, management, research, data, and integration challenges.

6.2. Gained Experience

Discuss technical and non-technical learning outcomes.

6.3. Conclusions

Summarize what the project achieved and why it matters.

6.4. Future Work

Mention possible future improvements and extensions.

References

[1] O. Obeid et al., “CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing,” in *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020, pp. 7022–7032.

[2] N. Habash, *Introduction to Arabic Natural Language Processing*. Morgan & Claypool Publishers, 2010.

[3] A. Abdelali, K. Darwish, N. Durrani, and H. Mubarak, “Farasa: A Fast and Furious Segmenter for Arabic,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 2016, pp. 11–16.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic (ARETA),” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021, pp. 21–32.

[5] W. Zaghouani, T. Zerrouki, and A. Balla, “SAHSOH@QALB-2015 Shared Task: A Rule-Based Correction Method of Common Arabic Native and Non-Native Speakers’ Errors,” in *Proceedings of the ACL-IJCNLP 2015 Student Research Workshop*, 2015, pp. 147–153.

[6] N. Habash and D. Palfreyman, “ZAEBUC: An Annotated Arabic-English Bilingual Writer Corpus,” in *Proceedings of the 13th Language Resources and Evaluation Conference*, 2022, pp. 780–790.

[7] A. Alrehili and A. Alhothali, “Tibyan corpus: balanced and comprehensive error coverage corpus using ChatGPT for Arabic grammatical error correction,” *PeerJ Computer Science*, vol. 11, e2724, 2025.

[8] J. Hajič, O. Smrž, P. Zemánek, J. Šnaidauf, and E. Beška, “Prague Arabic Dependency Treebank: Development in Data and Tools,” in *NEMLAR International Conference on Arabic Language Resources and Tools*, 2004.

[9] N. Habash et al., “CAMeL Treebank: An Open Multi-genre Arabic Dependency Treebank,” in *Proceedings of the 13th Language Resources and Evaluation Conference*, 2022, pp. 450–459.

[10] D. Halabi, E. Fayyoumi, and A. Awajan, “I3rab: A New Arabic Dependency Treebank Based on Arabic Grammatical Theory,” *arXiv preprint arXiv:2007.05772*, 2020.

[11] K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 2011, pp. 187–197.

[12] F. S. Al-Anzi et al., “Revealing the Next Word and Character in Arabic: An Effective Blend of Long Short-Term Memory Networks and CNN,” *Applied Sciences*, vol. 14, no. 10498, 2024.

[13] A. Taher, “Arabic Next Word Prediction using BERT and CBOW: A Comparative Study,” *Journal of Arabic Computational Linguistics*, 2024.

[14] C. Moukrim, T. Abderrahim, E. H. Benlahmer, and T. Almalki, “An innovative approach to autocorrecting grammatical errors in Arabic texts,” *Journal of King Saud University - Computer and Information Sciences*, vol. 33, no. 8, pp. 972–982, 2021.

[15] A. Solyman, Z. Wang, Q. Tao, A. A. M. Elhag, R. Zhang, and Z. Mahmoud, “Automatic Arabic Grammatical Error Correction based on Expectation-Maximization routing and target-bidirectional agreement,” *Knowledge-Based Systems*, vol. 240, 108103, 2022.

[16] B. Alhafni, N. Inoue, C. Khairallah, and N. Habash, “Advancements in Arabic Grammatical Error Detection and Correction: An Empirical Investigation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 600–618.

[17] B. Alhafni and N. Habash, “Enhancing Text Editing for Grammatical Error Correction: Arabic as a Case Study,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[18] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, “Character-Aware Neural Language Models,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2741–2749.

[19] E. Chirkunov, A. Al-Sabbagh, and N. Habash, “ARWI: Arabic Write and Improve — A Writing Assistant for Arabic Learners,” *arXiv preprint arXiv:2501.01234*, 2025.

[20] B. AlKhamissi, M. ElNokrashy, and M. Gabr, “Deep Diacritization: Efficient Hierarchical Recurrence for Improved Arabic Diacritization,” *arXiv preprint arXiv:2011.00538*, 2020.

[21] K. M. Almunirawi and R. S. Baraka, “Parsing Arabic Verbal Sentence Using Grammar Ontology,” *Journal of Engineering Research and Technology*, vol. 8, no. 2, 2021.

[22] W. Zaghouani, “Critical Survey of the Freely Available Arabic Corpora,” *arXiv preprint arXiv:1702.07835*, 2017.

Chapter A: Development Platforms and Tools

List programming languages, frameworks, libraries, operating systems, hardware, datasets, and development tools.

Chapter B: Use Cases

Describe system use cases, actors, preconditions, main flows, alternative flows, and expected results.

Chapter C: User Guide

This appendix should be included. Explain how to install, run, and use the final system. Add screenshots or step-by-step instructions when available.

Chapter D: Code Documentation

Document important modules, classes, APIs, configuration files, and code structure.

Chapter E: Feasibility Study

This appendix should be included. Add market, technical, operational, and financial feasibility details.